

Prompt Engineering

What is prompt engineering?

The process of crafting specific and clear instructions that guide generative AI models to produce accurate, relevant and desired outputs.

The techniques covered in here will be very useful for you in guiding the output of LLMs.

This isn't an exact science you may find some of the techniques we're going to cover work better than others in some cases and vice versa.

This isn't a set of concrete rules either, more of a guideline to help you get the most from AI.

Trying any of these can't hurt, iteration is the key to success!

Techniques

1. Assuming a role
2. Few shot learning
3. Chain of thought
4. XML tagging
5. DO and DON'T

Role prompting

Why use role prompting?

- Accuracy: assuming an expert role in the subject area can improve accuracy
- Tone: you can ask the model to respond in a certain tone / format
- Focus: assuming a role should encourage models to stay within the bounds of your task

Example

```
You are an expert Java software engineer that always follows best practices for the language and any libraries you use. You always write high quality, simple, testable, and loosely coupled code.
```

```
Can you create a template project for a Spring Web API?
```

When to use

- Creating agent files
- Focusing an LLM on specific task
- Keeping responses concise / to the point

Few shot learning

Lets you steer LLMs toward a new task by including a small amount of input / output examples.

Quick and relatively effective method of shaping model outputs.

```
Input:
<review id="1">
  I love this product, highly recommend you buy it!
</review>

Output:
<assistant_response id="1">
  Positive
</assistant_response>
```

When to use

- When you need rigid, specificly formatted output from the model

Chain of thought

Chain of thought prompting can force an LLM to 'think' about its response deeply. This can improve performance for larger, more complex tasks.

This encourages the model to approach a task in a step by step process

Example

Take a step by step approach, citing any relevant material. Walk me through your reasoning.

Do not make assumptions, ask me any refining questions you need.

When to use

When you need the model to handle a more complex task. If the model is missing a lot of edge cases.

XML Tagging

Structuring your prompt with XML tags to clearly declare intent. Tags can take whatever name you can think of.

```
You are an expert at reading financial statements.
```

```
<instructions>
```

```
Create a summary of all the expenses / income this company has had in the last 2 years.
```

```
</instructions>
```

```
<data>
```

```
{Data.xlsx}
```

```
</data>
```

```
<tone>
```

```
Use a professional tone, always get straight to the point of what you are trying to do.
```

```
</tone>
```

This can help you break down complex asks and keep the model focused on what you want to achieve.

When to use

Complex prompts with many parts

Do and Don't

As simple as it sounds, giving AI a list of DOs and DON'Ts can guide behaviour very well.

- DO: Read all instructions files present that are relevant to the ask
- DO: Ensure any code written has unit tests to cover it
- DON'T: Install any new libraries, use the functionality available in the current project
- DON'T: Break production

Simple, but effective

When to use

Guiding the model away from or to certain actions explicitly.

Best practices

You will find varying success with combinations of these, feel free to use any combination or none of them if they don't suit!

- Use a combination of techniques
- Plan your prompt before you write (Make a list of required actions etc...).
- Get straight to the point of what you are trying to do.

Meta prompting

Feeding your prompt back into AI and asking it to improve upon it based on your goals works surprisingly well! More complex models tend to do better at this.

This will hopefully help you create more useful prompts.

Questions?